
MACHINE LEARNING I

INSTRUCTOR: MOAYAD ALNAMMI

malnammi.github.io

Notice

- Slides prepared by Moayad Alnammi – malnammi.github.io

This deck is published at malnammi.github.io under CC BY-NC-SA 4.0. Reuse and adaptation require attribution to the author and the sources above; commercial use is not permitted; adaptations must carry the same license.

Outline

- Motivational practical examples of machine learning problems
- Definitions and mathematical notations
- What are the types of learning problems?
 - Supervised learning
 - Unsupervised learning
 - Reinforcement learning
- Resource recommendations

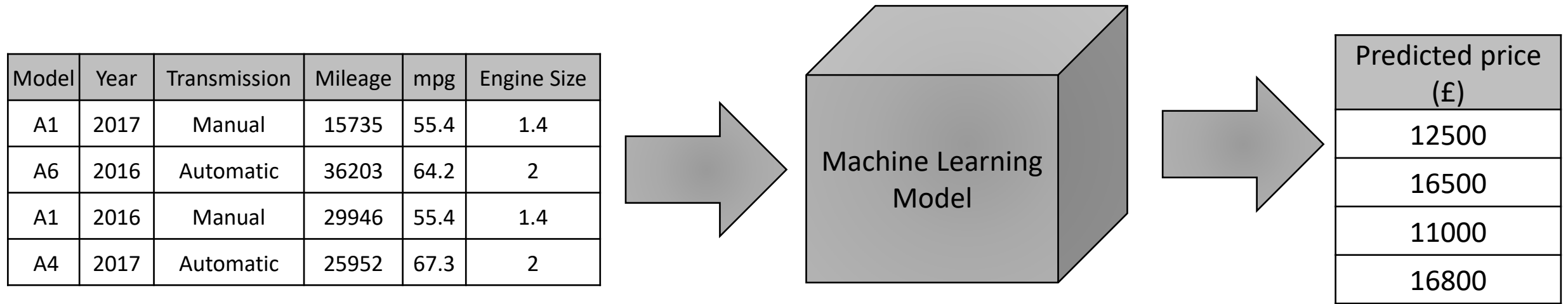
Definitions and Notations

Related Terms and Fields

- Artificial Intelligence
- Statistical learning and probability theory
- Pattern recognition
- Signal processing
- Data mining
- Information theory
- Optimization and linear algebra
- Business analytics

Price Prediction Example – Car Prices

- Task: given a car's description, predict its average market value.



Input is a list of car descriptors.
Each row represents a car.

See: www.kaggle.com/adityadesai13/used-car-dataset-ford-and-mercedes

What is Machine Learning? [1]

- We saw from the previous example that there is a **task**.
- The task has an **input** and an **output**.
- There is a desired goal for the task; e.g. **predict** an event with high **performance**.
- The machine learning model performs the task to achieve the desired goal with high performance.
- Performance measure is task-dependent (more on this in later lectures).

What is Machine Learning? [2]

- “Machine Learning is the study of algorithms that improve their **performance P** at some **task T** with **experience E**. A well-defined learning task is given by $\langle P, T, E \rangle$.” — Tom Mitchell. Machine Learning (Book). 1997.
- “Vast amounts of data are being generated in many fields, and the statistician’s job is to make sense of it all: to extract important patterns and trends, and understand “what the data says.” We call this learning from data.” — Elements of statistical learning (Book). 2012.
- Machine learning is the use of data to build a program that performs a specific task. The more data we have, the better the program gets at the task.

Learning from data – Car price prediction [1]

- **Task:** given a car's description, predict its average market value.
- **Performance** measure: **accuracy** of predicted value.
- **Experience:** Historical car price data. Suppose we have 1000 prices from past 6 months. Every row corresponds to an **example/sample/instances**.

Model	Year	Price	Transmission	Mileage	FuelType	Tax	mpg	Engine Size
A1	2017	12500	Manual	15735	Petrol	150	55.4	1.4
A6	2016	16500	Automatic	36203	Diesel	20	64.2	2
• • •								
A3	2019	17300	Manual	1998	Petrol	145	49.6	1

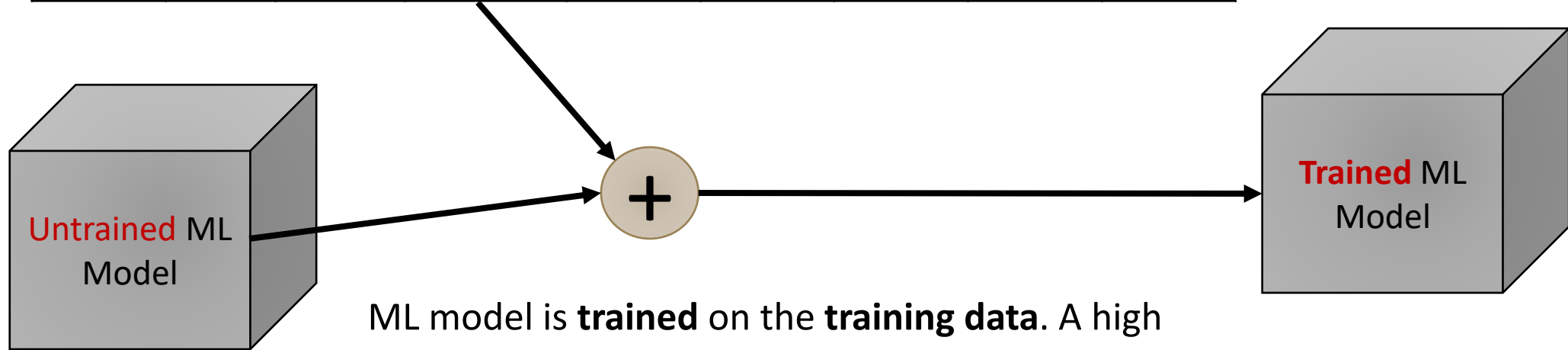
1000 **examples** to learn from.

Full dataset: www.kaggle.com/adityadesai13/used-car-dataset-ford-and-mercedes

Learning from data – Car price prediction [2]

Model	Year	Price	Transmission	Mileage	FuelType	Tax	mpg	Engine Size
A1	2017	12500	Manual	15735	Petrol	150	55.4	1.4
A6	2016	16500	Automatic	36203	Diesel	20	64.2	2
⋮								
A3	2019	17300	Manual	1998	Petrol	145	49.6	1

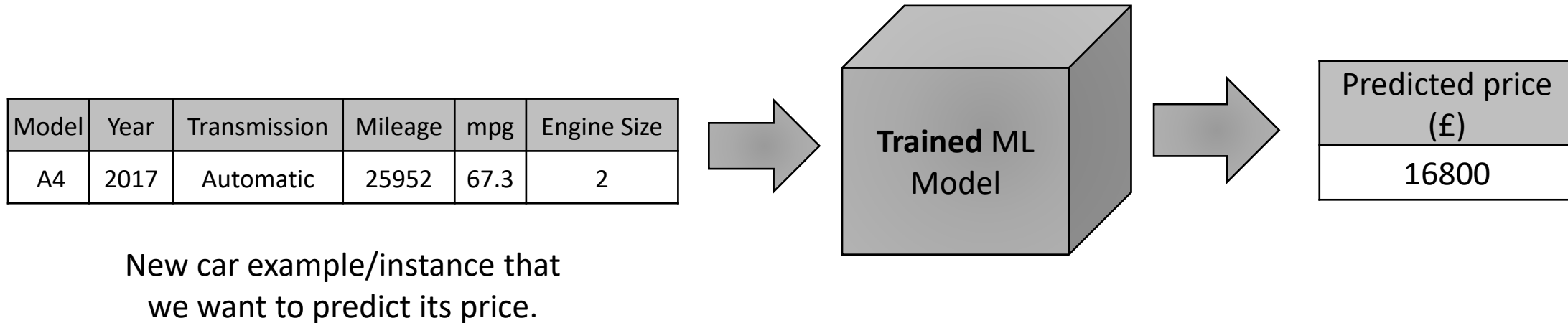
1000 **examples** to learn from.



ML model is **trained** on the **training data**. A high performing **Trained** model is produced.

Learning from data – Car price prediction [3]

- Trained ML model is used in production to predict on **new** instances. By new, we mean not part of the training data. So, the model has not been specifically trained on that instance.



What are features and labels?

- **Features**, input features, or input descriptors are the **inputs** to the machine learning model for a single example.
- **Labels** are the **output** of the ML model.
- Model, Year, Transmission, Mileage, mpg, and EngineSize are the **features** for the Car price prediction learning problem. The price is the **label**.
- Types of features (simplified): strings and numbers.

string	number	string	number	number	number
Model	Year	Transmission	Mileage	mpg	Engine Size
A4	2017	Automatic	25952	67.3	2

number
Predicted price (£)
16831

- All features and labels need to be represented in a number format to be processed by the model (computer program). More on this later.

Math notation for instances/examples

- On paper, we use mathematical notation for expressing instances and the learning problem.
- Common notation, we represent each instance as a fixed-length **vector** using the letter x .
- The labels are represented as a single-valued vector (scalar) using the letter y .

Model	Year	Transmission	Mileage	mpg	Engine Size
A4	2017	Automatic	25952	67.3	2

True price (£)
16800

- Thus, this instance is expressed as:

$x = [A4, 2017, Automatic, 25952, 67.3, 2]$ and $y = [16800]$

Math notation for a dataset

- A dataset is a set $\mathcal{S}$ with m instances (also called samples or examples).
- Each instance has a feature vector x and label y .
- $\mathcal{S} = \{ (x_1, y_1), (x_2, y_2), (x_3, y_3), \dots, (x_m, y_m) \}$

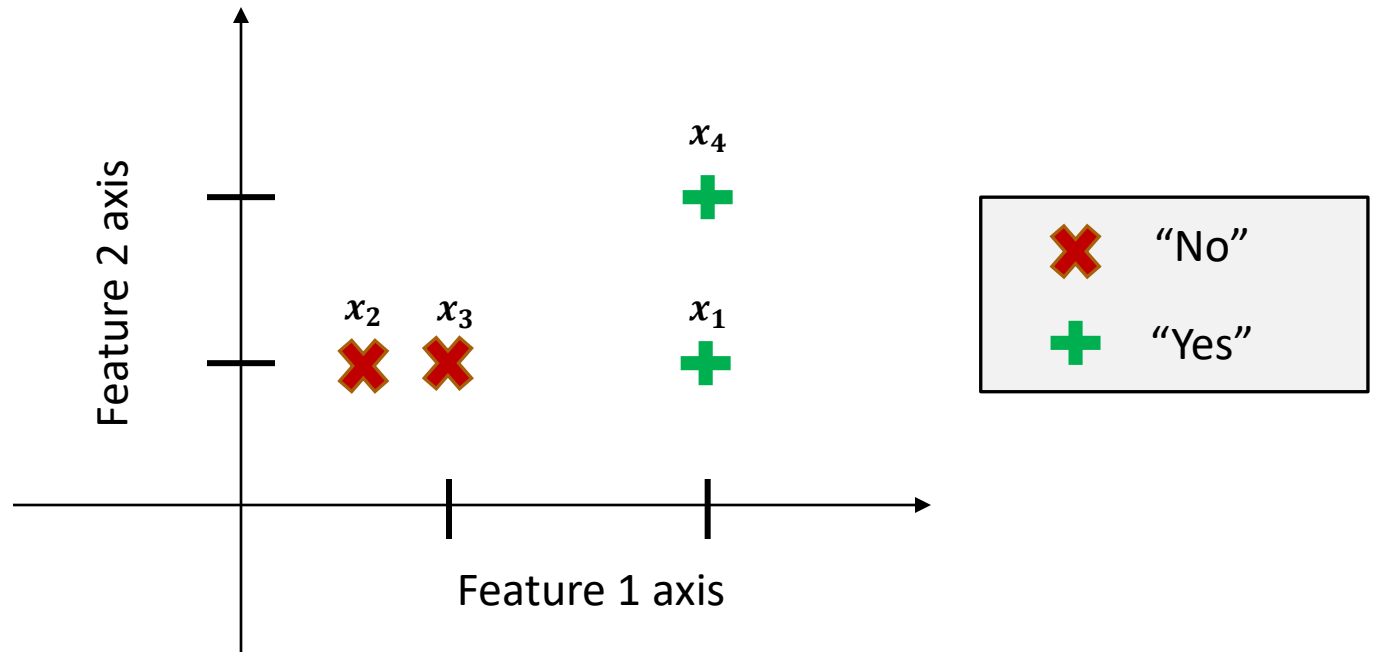
	Model	Year	Transmission	Mileage	FuelType	Tax	mpg	Engine Size		Price
x_1	A1	2017	Manual	15735	Petrol	150	55.4	1.4	y_1	12500
x_2	A6	2016	Automatic	36203	Diesel	20	64.2	2	y_2	16500
				.						.
				.						.
				.						.
x_m	A3	2019	Manual	1998	Petrol	145	49.6	1	y_m	17300

Represents a dataset. We can refer to this as a labeled dataset since the labels are known.

Feature space

- Using vector notation, the instances represent a point in n -dimensional cartesian space.
- For example, the below classification dataset has 2 features. Each instance represents a point in 2-dimensions.

	Feature 1	Feature 2	Label
x_1	2	1	Yes
x_2	0.5	1	No
x_3	1	1	No
x_4	2	2	Yes



Helpful Matrix Notation

- We can represent a dataset $\{(x_1, y_1), (x_2, y_2), \dots, (x_m, y_m)\}$ with m examples and n features as a feature matrix $X \in \mathbb{R}^{m \times n}$ and a label vector $y \in \mathbb{R}^m$.

$$X = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1n} \\ x_{21} & x_{22} & \dots & x_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ x_{m1} & x_{m2} & \dots & x_{mn} \end{bmatrix} = \begin{bmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_m^T \end{bmatrix} \quad \bigg| \quad Y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_m \end{bmatrix}$$

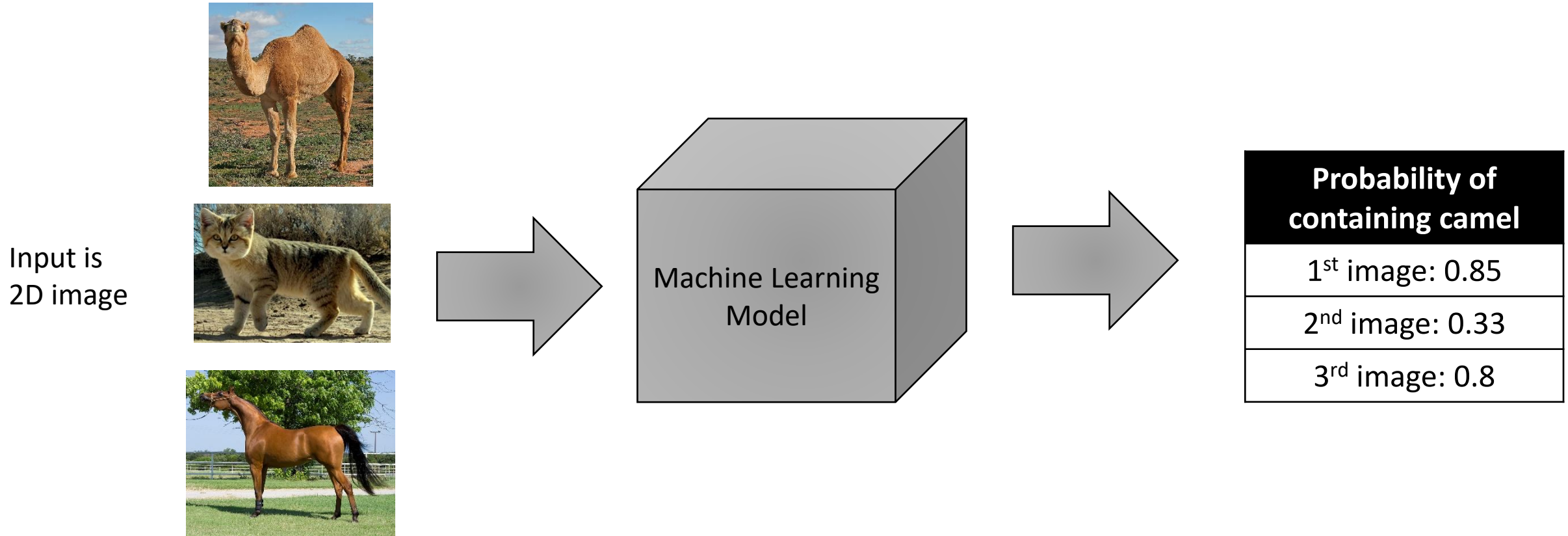
- X is referred to as the feature or data matrix. Y is the labels vector.

Machine Learning – Problem Types

- **Supervised Learning:** learning problem involves the use of input features to predict a label. Goal is to train a model to predict labels from input features.
- **Unsupervised Learning:** Learning problem does not have labels, only features. The goal is to analyze the data and find interesting patterns. We will see examples later.
- **Reinforcement Learning:**
 - Learning problem involves an agent that interacts with an environment. The agent can perform actions at every time step.
 - An agent's action results in a change in the environment.
 - The agent is rewarded or penalized based on the actions.
 - Through self-training in an environment and receipt of rewards, the agent gains experience and learns. With enough experience, the agent selects the best action given the situation.

SL Example – Image Recognition

- Task: given an image, predict if the image contains a camel.



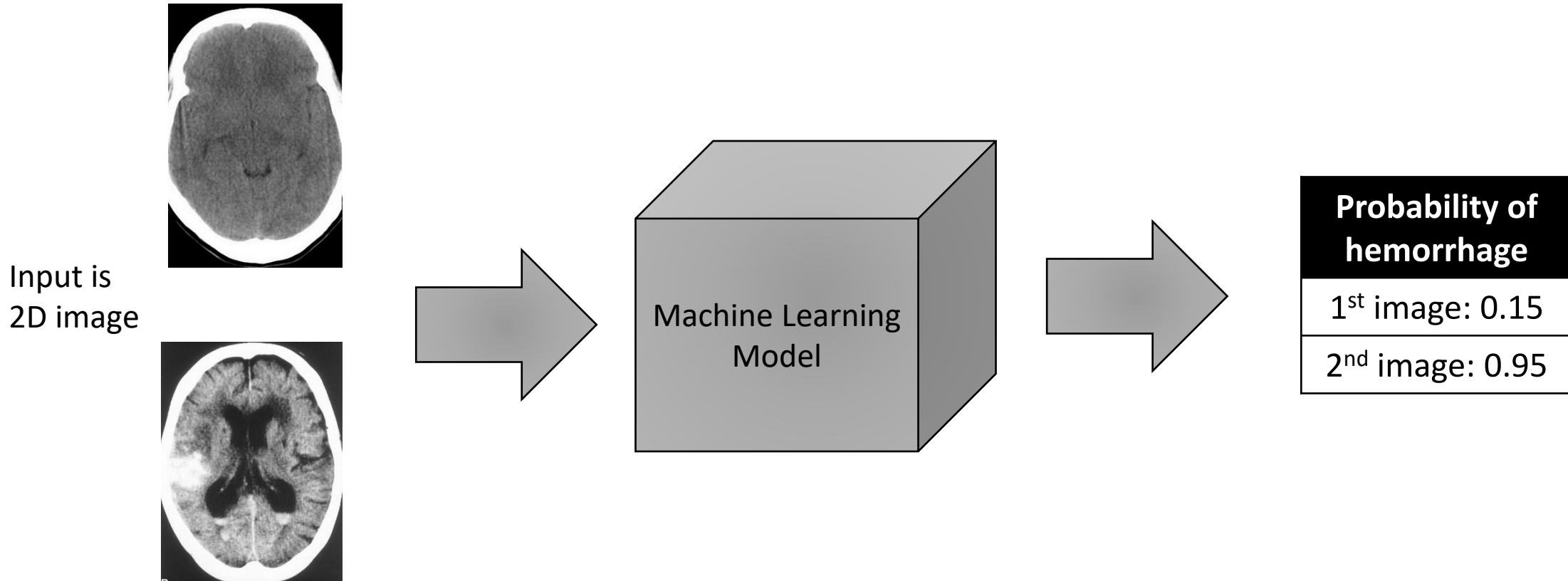
By Jjron - Own work, CC BY-SA 3.0, <https://commons.wikimedia.org/w/index.php?curid=2831408>

By Payman sazesh - camera trap, CC BY-SA 3.0, <https://commons.wikimedia.org/w/index.php?curid=24849743>

By Ealdgyth - Own work, CC BY-SA 3.0, <https://commons.wikimedia.org/w/index.php?curid=4165040>

SL Medical Example – Hemorrhage Diagnosis

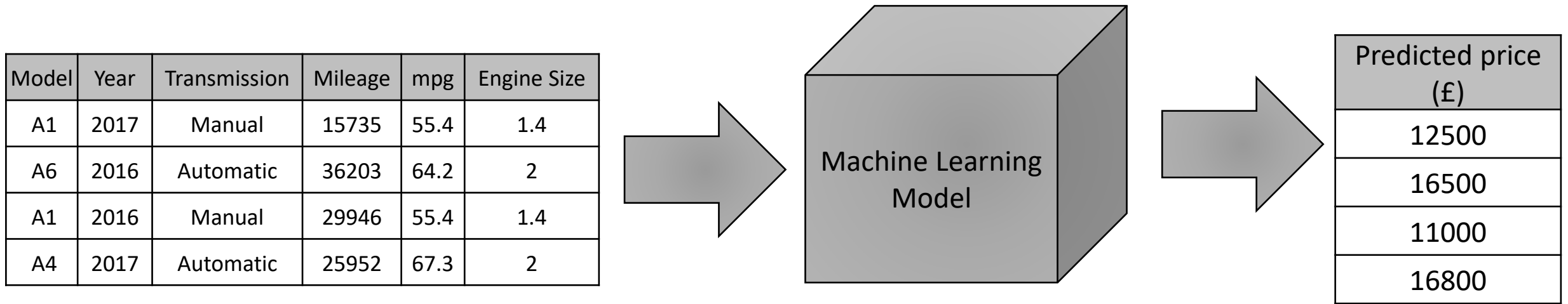
- Task: given a CT head scan, determine if there is hemorrhage.



See: www.kaggle.com/felipekitamura/head-ct-hemorrhage

SL Price Prediction Example – Car Prices

- Task: given a car's description, predict its average market value.

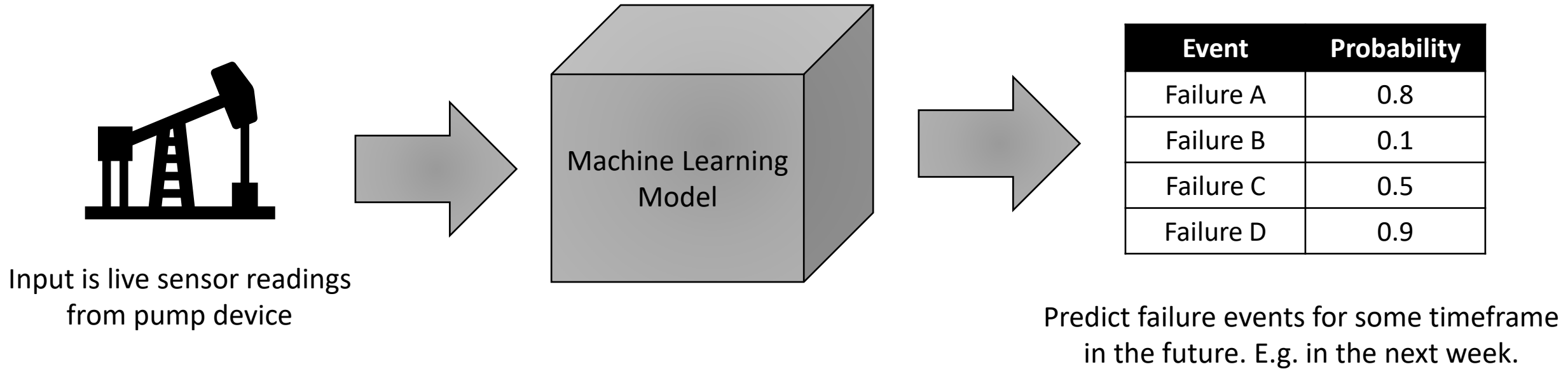


Input is a list of car descriptors.
Each row represents a car.

See: www.kaggle.com/adityadesai13/used-car-dataset-ford-and-mercedes

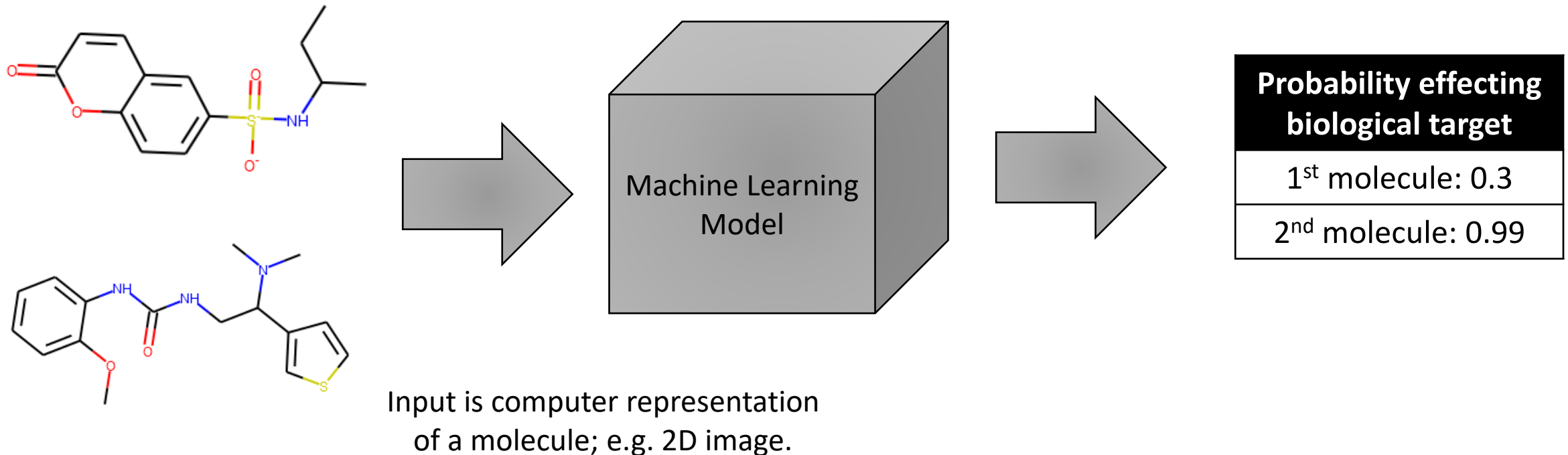
SL Oil & Gas – Detecting early failure of machines

- Task: given live sensor readings of a device, predict if a failure will occur in the future.



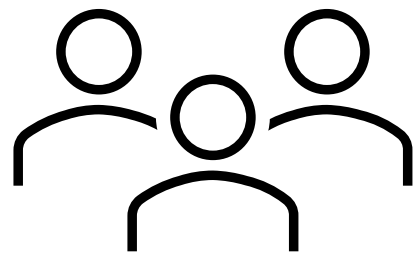
SL Drug Discovery Example – Finding new medicine

- Task: given a molecule, predict if it is effective against a biological target of interest. For example, we want to find molecules that work against a protein in the body that plays a role in a disease.

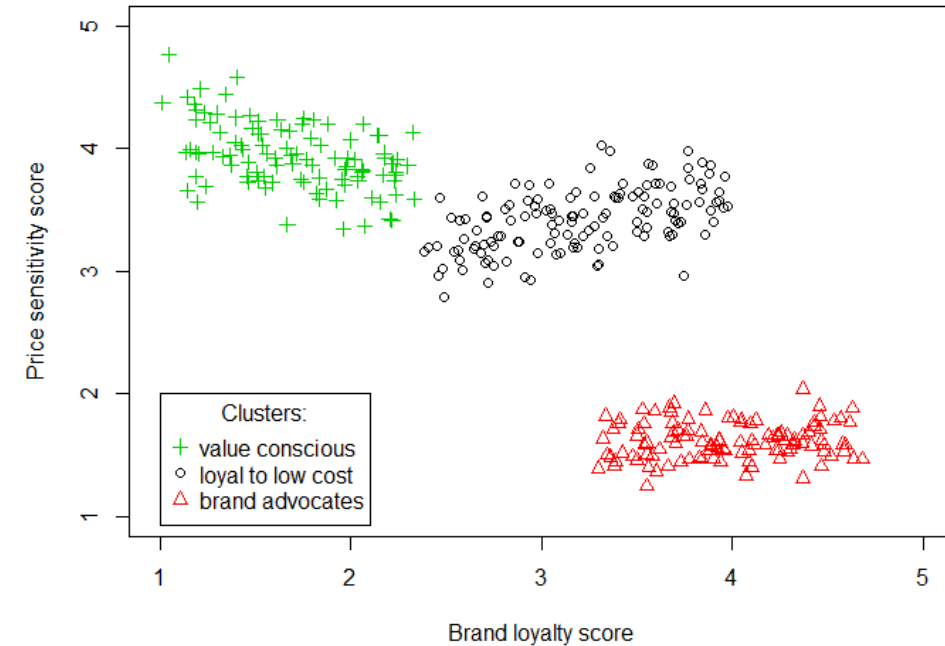
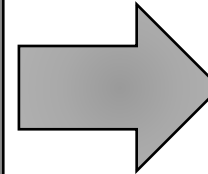
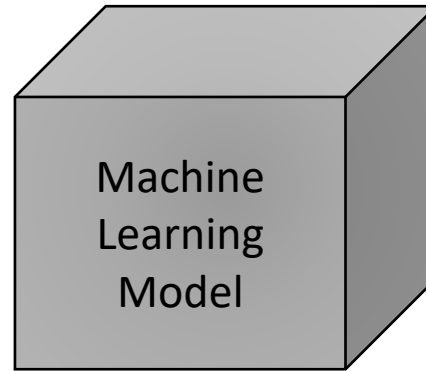
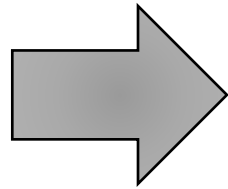


UL Example – Customer Segmentation/Clustering

- Task: given customer information, find interesting patterns to enhance business decisions. Notice there are no labels, just features.

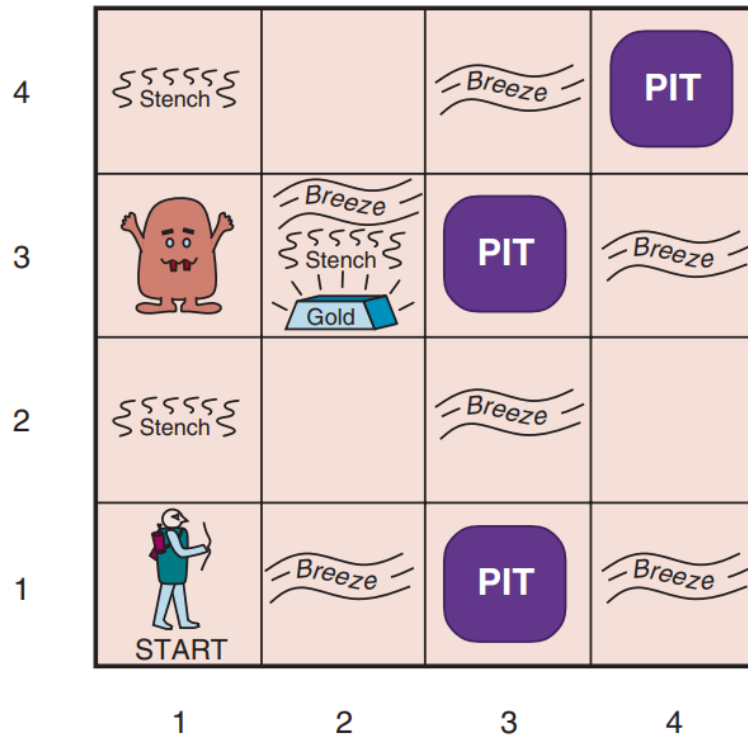


Customer data



RL Example – Learning Strategy for Environment

- Task: given a complex environment, find the optimal strategy for the robot agent.



Wumpus world toy problem

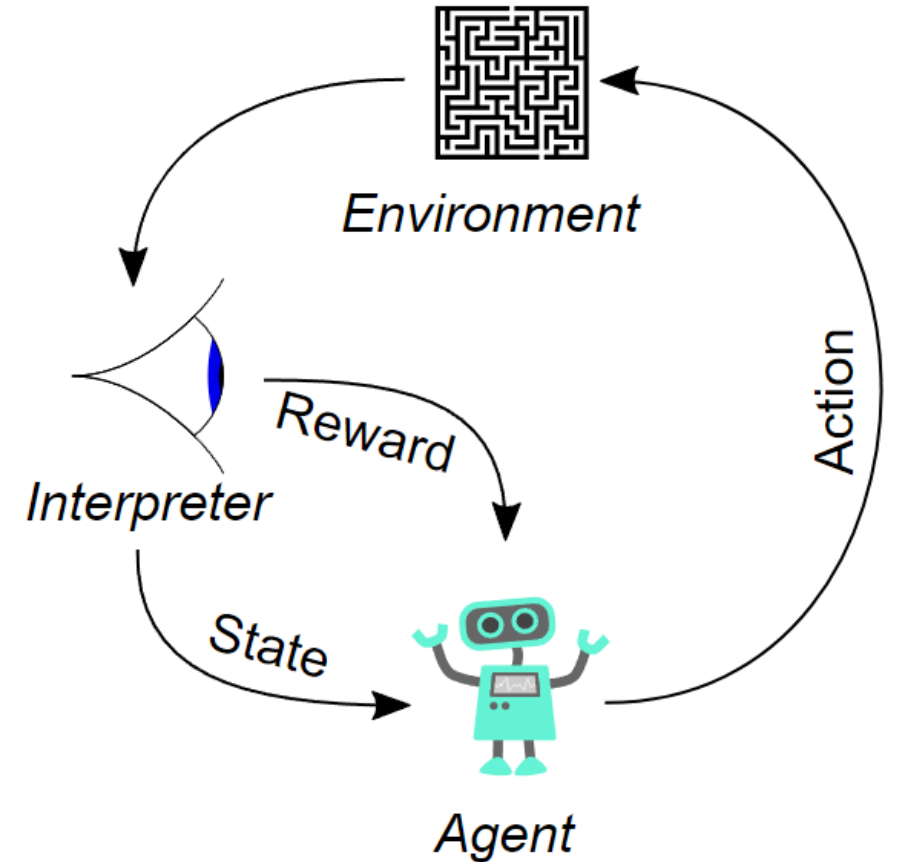


Image from [link](#)

Unsupervised Learning – Overview

Unsupervised Learning Overview [1]

The **unsupervised** learning problem consists of:

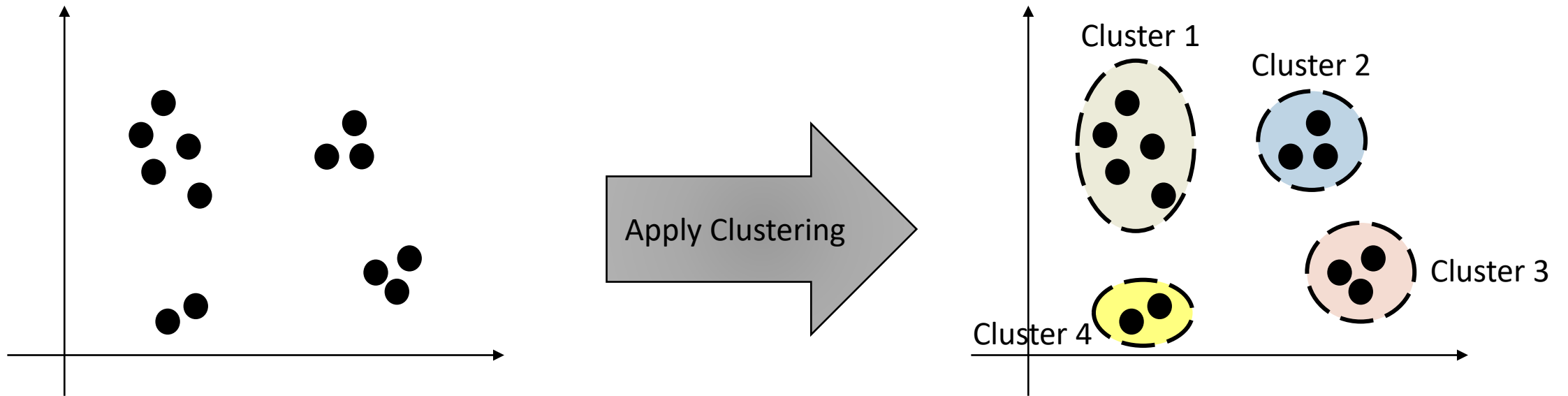
- **Unlabelled Dataset:** A dataset set $S = \{x_1, x_2, x_3, \dots, x_m\}$ consists of m instances (or examples).
- Note that we don't have any labels y .
- In practice, the dataset S is gathered from a real-world process.
- **Goal:** Find interesting patterns, structure, or groupings in the data.
- Common unsupervised tasks: clustering, anomaly detection, and dimensionality reduction.

Clustering (1)

- Given $\mathcal{S} = \{x_1, x_2, x_3, \dots, x_m\}$, group the instances into groups based on their distances to each other.
- How do we measure distance between x_i and x_j ? One simple way is via vector distance. Many other ways that is a function of the features.
- How many groups do we split the data into? Can fix this to a specific number, or make it adaptive based on a performance measure.

Clustering (2)

- Suppose we have a dataset with just 2 features and 13 instances. We want to cluster into 4 groups/clusters.

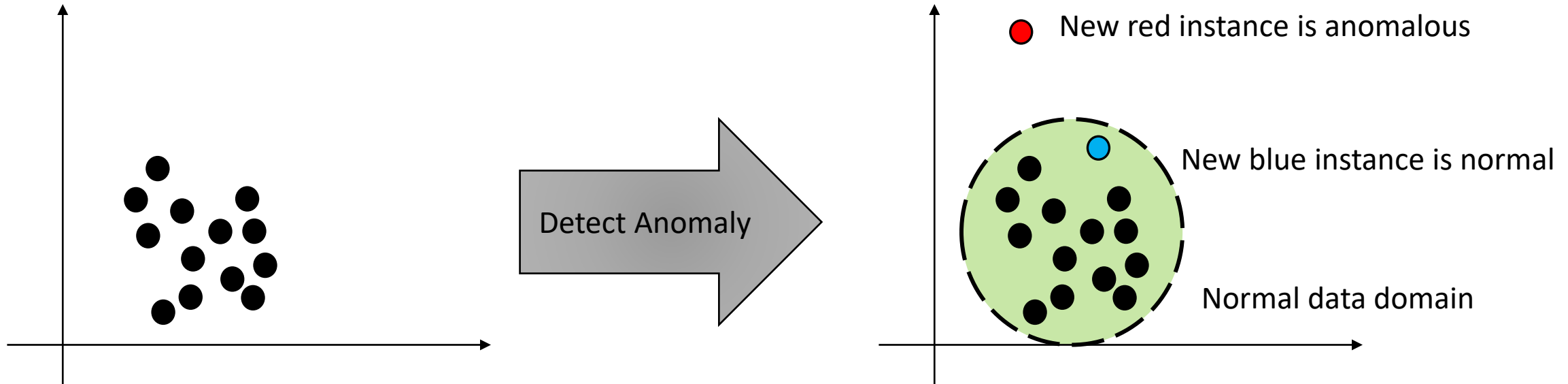


- Can analyze each cluster or group in more detail.
- Common application: visualization, consumer habits analysis, social network analysis.

Anomaly detection (1)

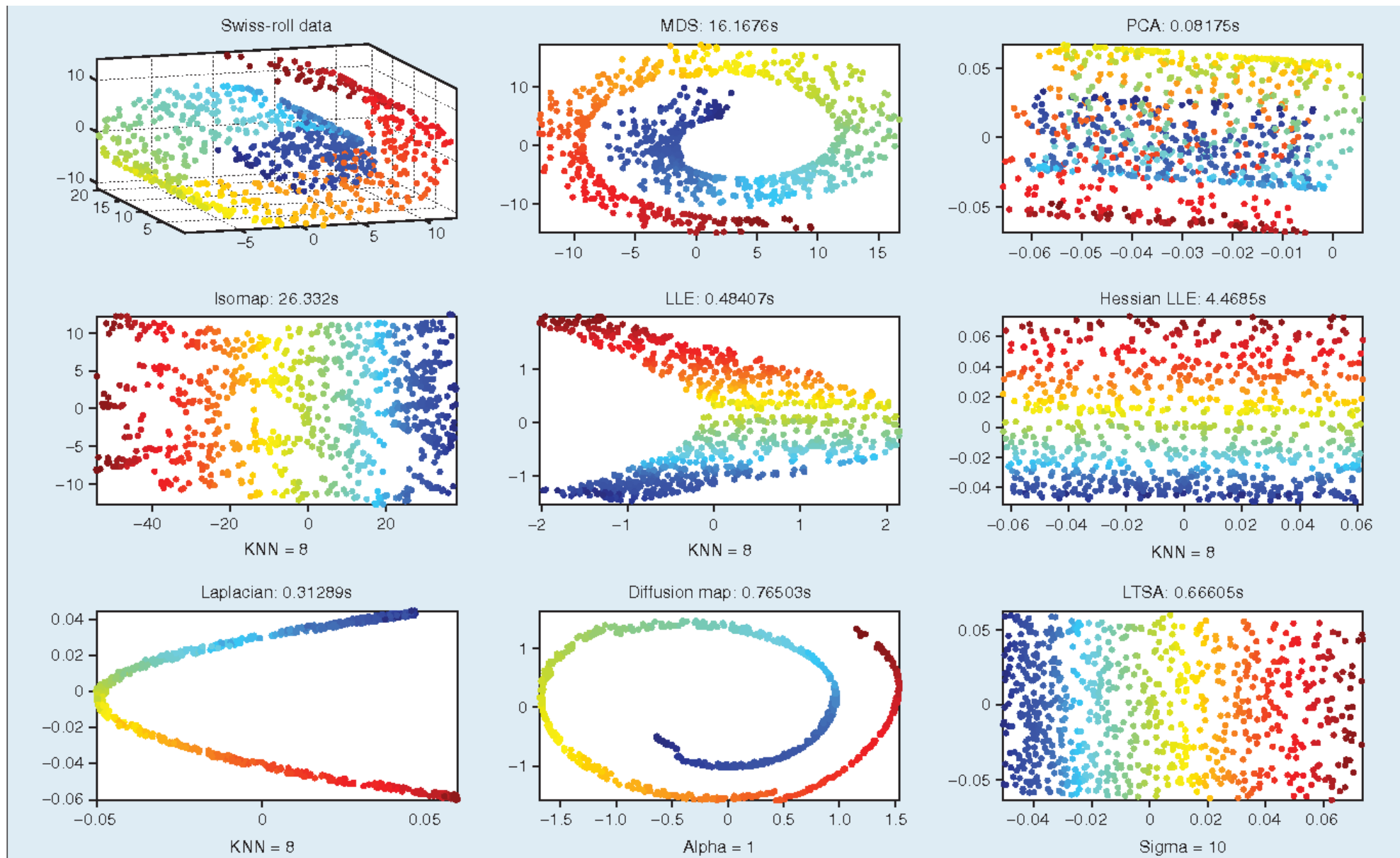
- Given $\mathcal{S} = \{x_1, x_2, x_3, \dots, x_m\}$, train a model to learn the **normal** pattern of an instance.
- The trained model can then be used to judge if a new instance is an outlier (anomaly, NOT normal, alarming).
- Can be used to detect alarming situations.
- Example 1: we have data on the purchasing habits of a person, we can detect if he purchases something that is out of the norm.
- Example 2: we have data from multiple sensors attached to oil pumps, we can detect if an oil pump's sensor readings are anomalous.
- Note that as we gather data over time, what is normal can shift. E.g. the purchasing habits of a person can change over time.

Anomaly Detection (2)



Dimensionality Reduction (1)

- Given $\mathcal{S} = \{x_1, x_2, x_3, \dots, x_m\}$, where x_i has d features, reduce the number of features from d to k .
- Typically, the data is mapped to lower dimensions while preserving the *structure* or *distance* of instances to each other.
- Common application is for visualization. That is map, the dataset from d dimensions to 2 or 3 dimensions and plot.
- The higher the dimensions, the harder it is to reduce to 2 or 3 dimensions while preserving the structure of the data.
- Other common application is to reduce the memory size of a high dimensional dataset. Fewer features -> smaller memory size.



3D Swiss-roll data (top-left) is reduced to 2D using eight dimensionality reduction techniques.

Image from: <https://ieeexplore.ieee.org/document/5401149>

Supervised Learning

Simplified

Supervised Learning Overview [1]

The supervised learning problem consists of:

- **Labeled Dataset:** A dataset set $S = \{(x_1, y_1), (x_2, y_2), (x_3, y_3), \dots, (x_m, y_m)\}$ consists of m instances (or examples).
- **Features:** Each x_i for $i = 1, 2, 3, \dots, m$ contains information about the instance. E.g. numbers, strings, etc.
- **Labels:** Each y_i for $i = 1, 2, 3, \dots, m$ is **assumed** to be a **function** of x_i . That is:

$$y_i = f(x_i)$$

- In practice, the dataset S is gathered from a real-world process.

Supervised Learning Overview [2]

The supervised learning problem consists of (cont.):

- **Machine Learning Model:** A machine learning model that produces a function h that is an **approximation** of the labeling function f .
- Question: Why doesn't the ML model just set $h = f$?
- Answer: The labeling function f is **unknown**.
- **Goal:** The ML model uses the labeled dataset S to produce $h \approx f$.

Supervised Learning with an example [1]

	Model	Year	Transmission	Mileage	FuelType	Tax	mpg	Engine Size		Price
x_1	A1	2017	Manual	15735	Petrol	150	55.4	1.4	y_1	12500
x_2	A6	2016	Automatic	36203	Diesel	20	64.2	2	y_2	16500
				•						•
				•						•
				•						•
x_m	A3	2019	Manual	1998	Petrol	145	49.6	1	y_m	17300

1. Suppose this labelled dataset $\mathcal{S}$ of $m = 1000$ instances was gathered from the real-world.

2. Suppose there is an **unknown** labelling function f such that:

$$y = f(\text{Model}, \text{Year}, \text{Transmission}, \text{Mileage}, \text{FuelType}, \text{Tax}, \text{mpg}, \text{EngineSize})$$

More specifically:

$$y_i = f(x_i) \text{ recall that } x_i \text{ is a vector. Note there are 8 column features for this specific dataset.}$$

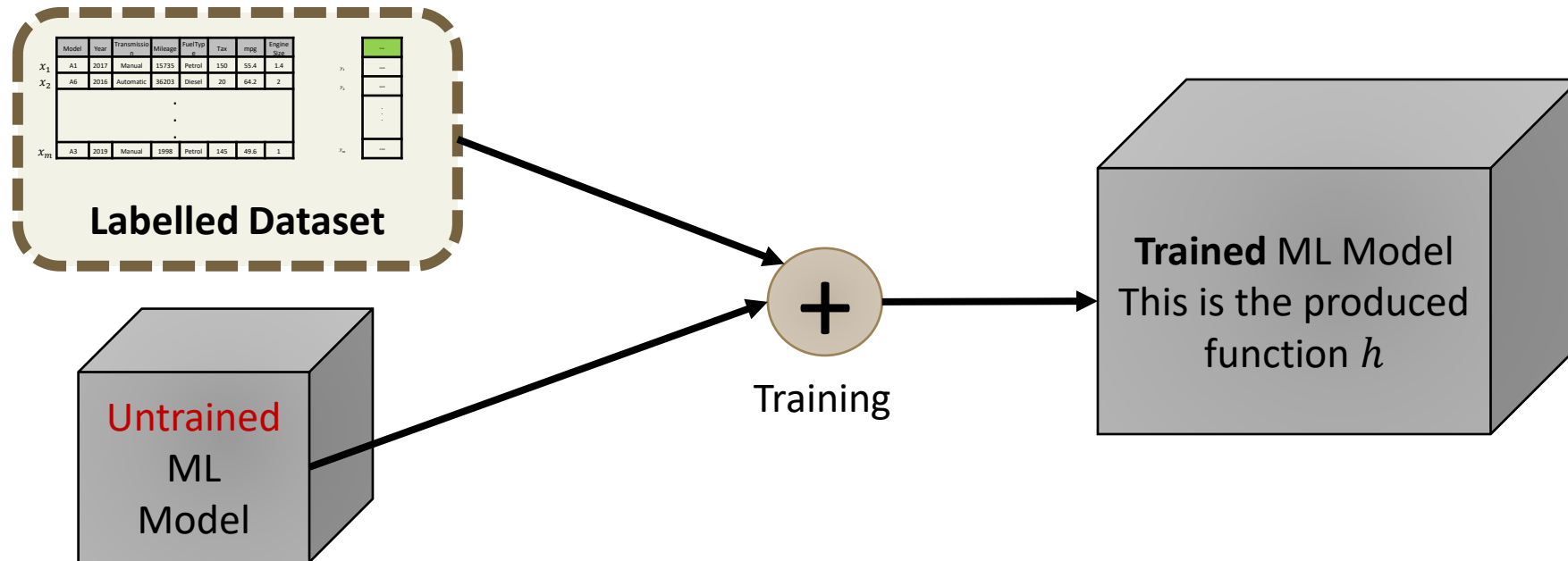
$$= f(x_{i1}, x_{i2}, x_{i3}, x_{i4}, x_{i5}, x_{i6}, x_{i7}, x_{i8}) \text{ where } x_{ij} \text{ denotes the } j\text{th feature value of the } i\text{th instance.}$$

Supervised Learning with an example [2]

So, we have this labelling function whose form or format is unknown:

$$y_i = f(x_{i1}, x_{i2}, x_{i3}, x_{i4}, x_{i5}, x_{i6}, x_{i7}, x_{i8})$$

3. The goal of the ML model is to use this labelled dataset to produce a function h that approximates f .



4. Now that we have h , then we can predict the label of new instances: $h(x_{new})$. Such that: $h(x_{new}) \approx f(x_{new})$

The Two Types of Supervised Learning

- **Regression:** The output labels are **continuous** real values. E.g. prices, scores, sensor readings, temperature, etc.
- **Classification:** The output label is a value from a small finite set.
 - **Binary classification:** output label is assigned a single value from a set of size 2. For example {No, Yes} or {0, 1}.
 - **Multiclass classification:** output label is assigned a *single* value from a set of size 3 or more. For example {small, medium, large} or {left, right, forward, down}.
 - **Multilabel classification:** output label can be assigned *multiple* values from a set of size 2 or more. For example, when detecting multiple failure events for a single device; a device can suffer from multiple failures (see [slide](#)).

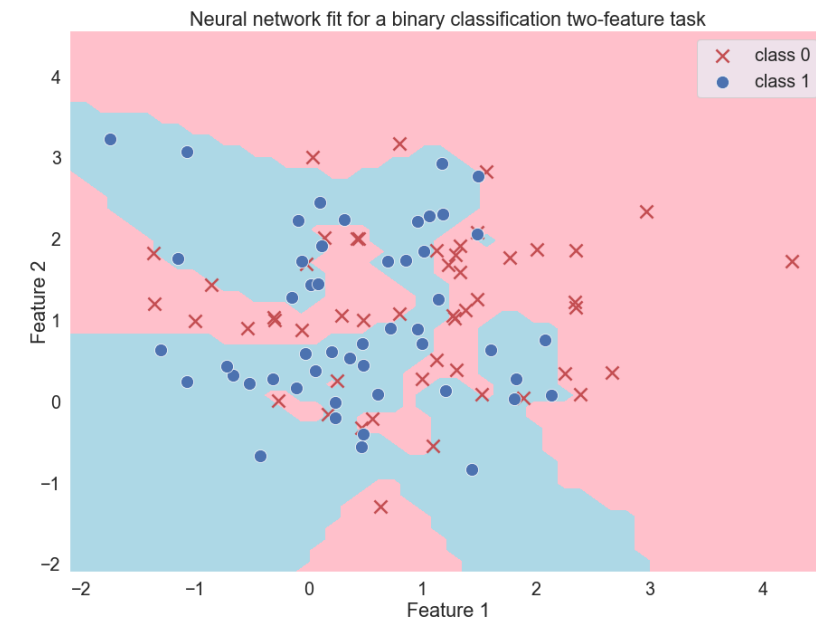
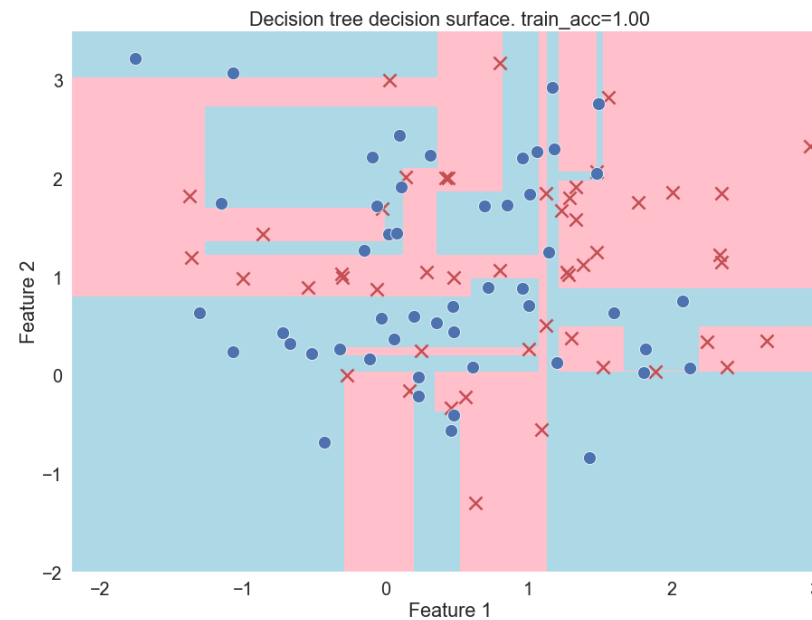
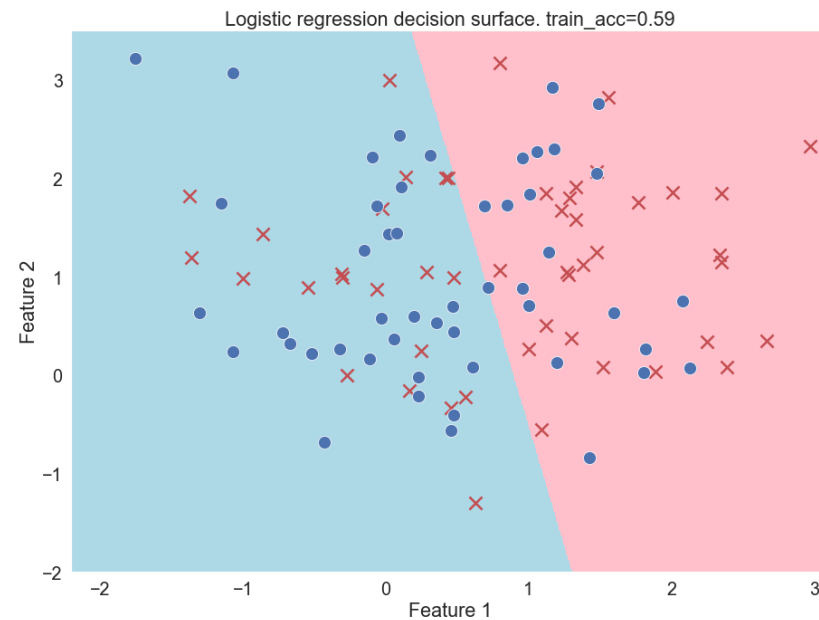
Recap so far...

- **Labelled dataset** gathered from real-world process.
- Machine learning model that **trains** on the labelled dataset to produce an approximator function h .
- New data without labels can now be passed to the trained ML model to compute predictions: $h(x_{new})$.
- Supervised learning types: regression and classification.
- But how do we know if our trained model will perform well in practice? It may have learned a *bad* approximator function.

Supervised Learning Goal – Generalization

- Given a **labelled** dataset set $S = \{(x_1, y_1), (x_2, y_2), (x_3, y_3), \dots, (x_m, y_m)\}$, the goal in supervised learning is to use the data to train a machine learning model that is able to **generalize** to **unseen** instances.
- **Generalization:** The ability of a model to **correctly** predict the label of *new* instances that it did not train on.

- Every ML algorithm will make assumptions on the *form* of the labelling function f .
- Here are some famous ML algorithms on their assumptions:
 - Linear/Logistic regression: assumes f is a linear function of the features.
 - Decision tree: assume f is axis-aligned rectangles decision boundaries.
 - Neural network: assume f is highly non-linear function of the features.
- See the following 2-dimensional classification training dataset where Logistic regression, decision tree, and neural network is fit. Notice that *form* of these functions. Notice the accuracy of the training set.
- You may think that you should always use the most powerful/complex models to fit datasets, however, we will see this is not a good idea.



Training an ML algorithm [1]

- Every ML algorithm will make assumptions about the labelling function f .
- For example, let $\mathcal{H} = \{h_1, h_2, h_3, \dots, h_{100}\}$ be a finite set of 100 hypotheses that some ML algorithm considers.
- How do we find the hypothesis in $\mathcal{H}$ that best approximates f ?
- We can use a loss function ℓ with the data S to find $h \in \mathcal{H}$ that gives the smallest error. This is called **training** on the data S .
- Three common loss functions:

$\ell(h(x_i), y_i) = |h(x_i) - y_i|$ absolute value difference; known as L-1 norm.

$\ell(h(x_i), y_i) = (h(x_i) - y_i)^2$ squared value difference; known as L-2 norm.

$\ell(h(x_i), y_i) = \begin{cases} 0 & \text{if } h(x_i) == y_i \\ 1 & \text{if } h(x_i) \neq y_i \end{cases}$ known as 0-1 loss.

Training an ML algorithm [2]

- We can use a loss function ℓ with the data S to find $h \in \mathcal{H}$ that gives the smallest error. This is called **training** on the data S .
- Given a labelled dataset S , hypothesis h , and loss function L . We can compute the **empirical loss** of h as following:

$$\begin{aligned} EmpLoss(S, h) &= \frac{1}{|S|} \sum_{(x_i, y_i) \in S} \ell(h(x_i), y_i) \\ &\approx \frac{1}{m} \sum_{i=1}^m \ell(h(x_i), y_i) \end{aligned}$$

This is the avg loss of h on the labelled dataset S .



- How do we find the hypothesis in $\mathcal{H}$ that best approximates f ? We select the hypothesis that gives the minimum empirical loss. This is called training (also known as empirical risk minimization).

$$h^* = \arg \min_{h \in \mathcal{H}} EmpLoss(S, h)$$

Training an ML Algorithm [3]

- Suppose that $\mathcal{D}$ is the probability distribution for the type of data we will see in production. Then what we really want to minimize is the **true loss or generalization loss**:

$$\begin{aligned} \text{TrueLoss}(h) &= E_{x \sim \mathcal{D}}[\ell(h(x), y)] \\ &= \int_x \ell(h(x), y = f(x)) p(x) dx \end{aligned}$$

$$h^* = \arg \min_{h \in \mathcal{H}} \text{TrueLoss}(h)$$

← Since x is distributed according to $\mathcal{D}$, we can compute the expectation $E[\cdot]$. Which is the average loss of h on all **possible** instances x .

← This is explicitly an integral over the possible values of x , weighed by the prob. of x . Some instances may appear more frequently than others.

- In practice, we don't know the underlying distribution $\mathcal{D}$. But we have access to a **sample** of this underlying distribution in the form of S .
- Thus, we **can** compute the Empirical Loss on S , but **not** the True Loss (Generalization) of the underlying distribution.

Empirical Risk vs True Risk (generalization)

$$TrueLoss(h) = E_{x \sim \mathcal{D}}[\ell(h(x), y)] \qquad EmpLoss(S, h) = \frac{1}{|S|} \sum_{(x_i, y_i) \in S} \ell(h(x_i), y_i)$$

- Where did the samples of S come from? From the underlying distribution.
- How good of an estimate is the empirical loss? Recall the law of large numbers from statistics.
 - The sample average of n samples $s_1, s_2, \dots, s_n$ converges to the expectation (mean) $E[s]$ as $n \rightarrow \infty$

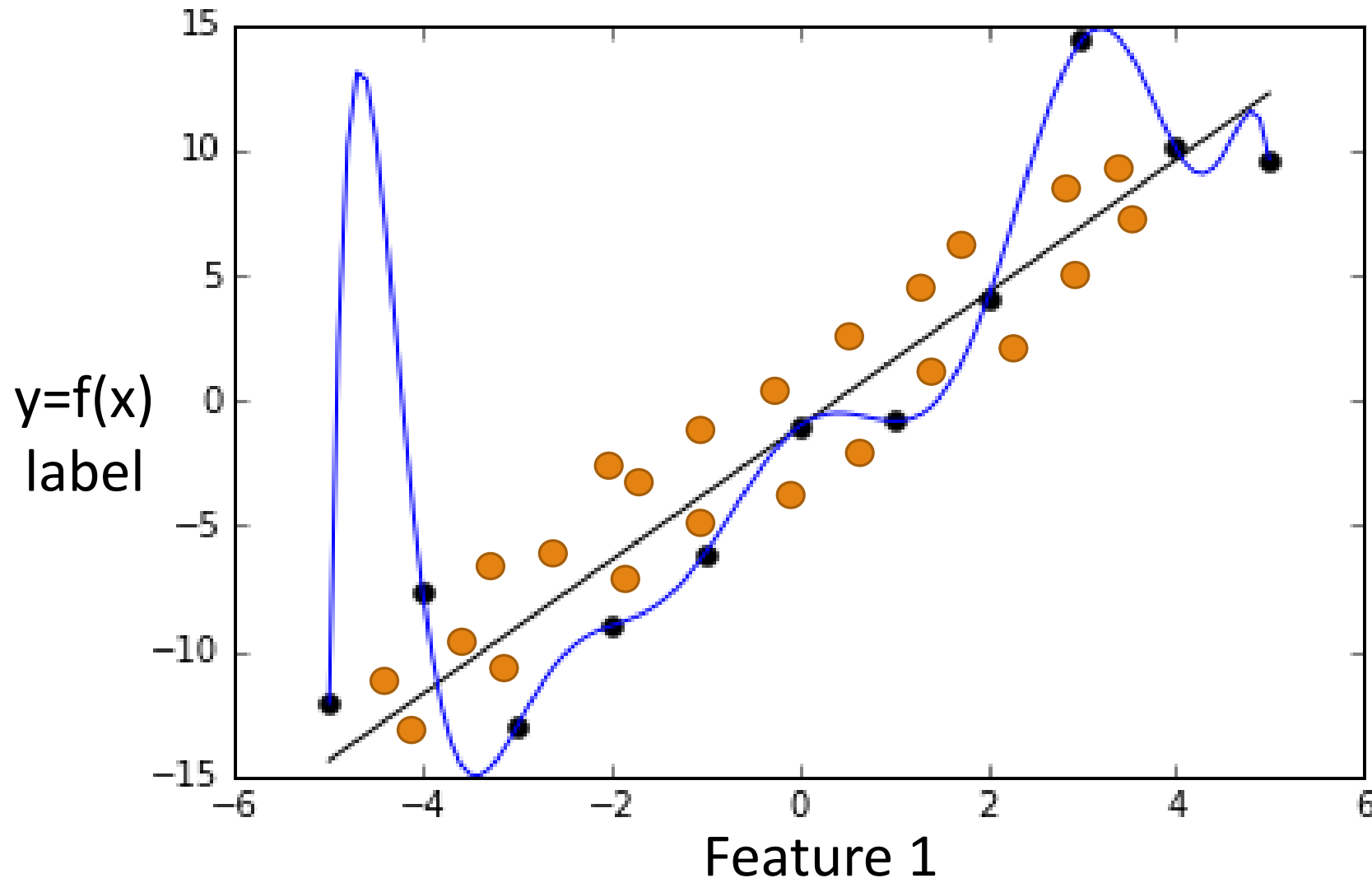
For any h , $EmpLoss(S, h) \rightarrow TrueLoss(h)$ as $|S| \rightarrow \infty$

$$\arg \min_{h \in \mathcal{H}} EmpLoss(S, h) \rightarrow \arg \min_{h \in \mathcal{H}} TrueLoss(h) \text{ as } |S| \rightarrow \infty$$

- So, what we can say is that empirical loss minimization is a sound strategy.
 - As the size of the dataset increases, the empirical loss approaches the true loss.
 - With large dataset, minimizing empirical loss is approximate to minimizing true loss.
- What can happen if dataset is not large enough?

Overfitting [1]

An ML algorithm with $\mathcal{H} = \{h_{blue}, h_{black}\}$



1. Suppose we have a learning problem with 1 input feature. We have a labelled dataset S. We plot the points (x, y) that are in S. Assume we use distance to compute the loss.

2. Suppose S are the black points. This is the data we have.

3. The orange points represents the underlying data. We don't have this data. It is hidden/unseen.

4. The blue hypothesis clearly predicts the points in S with zero empirical loss. But it has high true loss. This is a case of **overfitting**.

5. However, the black line hypothesis is a **better fit** for the true loss.

Overfitting [2]

$$h^* = \arg \min_{h \in \mathcal{H}} EmpLoss(S, h)$$

- In the previous plot. We saw that the blue hypothesis h^* clearly minimizes the loss on S (empirical loss). But has high loss on the underlying unseen data distribution (true loss). This happens when the machine learning algorithm **overfits** (memorizes) the data it trained on.
- Overfitting generally occurs because:
 - During training, we find the hypothesis h^* with minimum EmpLoss on the specific training dataset S . This is not necessarily the minimizer of the TrueLoss.
 - We say that empirical loss minimizer h^* is **biased** towards the training dataset.
 - If dataset S is not large enough we get a wide gap ϵ between EmpLoss and TrueLoss.

$$|EmpLoss(S, h^*) - TrueLoss(h^*)| = \epsilon$$

- **Main Takeaway:** it is *generally* not safe to rely on $EmpLoss(S, h^*)$ value to determine if model is good.
- What is alternative? And how can we detect overfitting?
 - We searched through $\mathcal{H}$ to find the best h^* for S , so performance on S *tends* to be good (**biased** estimate).
 - We can use another dataset Z that was not used for training. This is typically called the **validation set** or **test set**.
 - Compute $EmpLoss(Z, h^*)$. This is an **unbiased** estimate of $TrueLoss(h^*)$.
 - You can detect overfitting if $EmpLoss(S, h^*)$ is low, but $EmpLoss(Z, h^*)$ is high.

Training and Test sets

- Suppose we have gathered a labelled dataset S of size m from a real-world process.
- Split the dataset S into two sets: Training set T and test set Z .
- Note: other names for test set are validation set and held-out set. Some ML references denote the validation set as being different from test set (so you have three sets: training, validation, and test).
- Suppose we use an ML algorithm that considers a hypothesis space with 100 functions $\mathcal{H} = \{h_1, h_2, \dots, h_{100}\}$
- Train the algorithm to select from $\mathcal{H}$ the loss minimizer h^* on the training set T :

$$h^* = \arg \min_{h \in \mathcal{H}} EmpLoss(T, h)$$

Note use of
training set

- Note that since the training set T was used to select the hypothesis from $\mathcal{H}$. The empirical loss of h^* on T is biased to be low. This loss is called the **training loss**.
- So, we need another labelled dataset that has not been used in the training. We can use the test set Z . This is the **test loss**, and it is an *unbiased* (good) estimator of the true loss.

$$EmpLoss(Z, h^*) \approx TrueLoss(h^*)$$

Supervised Learning Summary

- The goal of supervised learning is generalization.
- We do not know underlying distributions. But have access to *sampled* dataset $S = \{(x_1, y_1), \dots, (x_m, y_m)\}$
- Most ML algorithms will make assumptions on the relationship between x and y . These assumptions give rise to a hypothesis space $\mathcal{H}$. Some hypothesis spaces are more complex than others (able to learn complex decision boundaries/surfaces).
- When training, the ML algorithm will minimize some loss function over the training set and find $h^* \in \mathcal{H}$. This is called empirical loss minimization (ERM).
- The training loss is not a good estimate of the true loss.
- Unless you have a sufficiently large training set that covers the underlying data distribution, overfitting can occur if we rely on the training loss.
- So, it is recommended, that you use another dataset's loss (not the training set). We call this test set.

- A hypothesis h is said to **overfit** a training set, if the training set loss is low ($EmpLoss(T, h)$ is low) and the true loss is high ($TrueLoss(h)$ is high).
 - A practical way to detect this is if the training set loss is low and the test set loss is high.
- A hypothesis h is said to **underfit** a training set, if the training set loss is high ($EmpLoss(T, h)$ is high) and the true loss is high ($TrueLoss(h)$ is high).
 - A practical way to detect this is if the training set loss is high and the test set loss is high.
- The notions of *high* and *low* is set by you depending on your problem. You can formalize it by specifying an acceptance tolerance/threshold value ϵ .

Practical Consideration – Splitting, training, and evaluating

Splitting the dataset Train-Test set

- Suppose your labelled dataset is S .
- You want to split S into training set T and test set Z .
- A large training set gives more examples to learn from.
- A large test set gives better estimate of true loss/performance.
- Rule of thumb in practice: split S such that train is 80% and test is 20%.
E.g. for 100 examples, train is 80 examples and test is 20 example.
- How should we split the data?
 - Regression task: split should be **uniformly random**.
 - Classification task: The split should **stratified random**.

Splitting the dataset Train-Test: Regression Task

Labelled dataset

$$\mathcal{S} = \{ (x_1, y_1), (x_2, y_2), (x_3, y_3), \dots, (x_{100}, y_{100}) \}$$

80-20 split
uniformly
random

How to split uniformly at random?

In python:

```
np.random.permute(100)
```

Training dataset randomly selects 80 instances

$$\mathcal{T} = \{ (x_{95}, y_{95}), (x_{12}, y_{12}), (x_{45}, y_{45}), \dots, (x_{71}, y_{72}) \}$$

Test dataset randomly selects 20 instances

$$\mathcal{Z} = \{ (x_{81}, y_{81}), (x_{65}, y_{65}), \dots, (x_{32}, y_{32}) \}$$

- Note that these two sets are disjoint. In other words, no instances from $\mathcal{Z}$ are in $\mathcal{T}$, and vice-versa.

Splitting the dataset Train-Test: Classification Task

Labelled dataset with 30 **+** labels and 70 **-** labels.

$$\mathcal{S} = \{ (x_1, +), (x_2, -), (x_3, -), \dots, (x_{100}, +) \}$$

80-20 split
stratified
random

Split randomly, but try to preserve the same percentage of **+** and **-**. Why?

How to do this in python?

Training dataset randomly selects 80 instances

Ensure there are 24 **+** labels and 56 **-** labels

$$\mathcal{T} = \{ (x_{95}, -), (x_{12}, +), (x_{45}, +), \dots, (x_{71}, -) \}$$

Test dataset randomly selects 20 instances

Ensure 6 **+** labels and 14 **-** labels

$$\mathcal{Z} = \{ (x_{81}, +), (x_{65}, +), \dots, (x_{32}, -) \}$$

- Note that these two sets are disjoint. In other words, no instances from $\mathcal{Z}$ are in $\mathcal{T}$, and vice-versa.

Splitting, training and evaluating using [scikit-learn](#)

- Suppose your labelled dataset is S consists of 1000, and you want to split into 80% training and 20% testing.
- See [link](#)

```
1 import numpy as np
2 from sklearn.model_selection import train_test_split
3 from sklearn.linear_model import LinearRegression
4 from sklearn.metrics import accuracy_score
5
6 # stratify split the dataset X, y into 80-20 train-test
7 X_train, X_test, y_train, y_test = train_test_split(X, y,
8                                                     test_size=0.2,
9                                                     stratify=y,
10                                                     random_state=42)
11
12 # train a linear regression model
13 model = LinearRegression()
14 model.fit(X_train, y_train)
15
16 # evaluate accuracy on test set
17 y_pred_on_test = model.predict(X_test)
18 test_acc = accuracy_score(y_true=y_test, y_pred=y_pred_on_test)
19 print('Test accuracy: {}'.format(test_acc))
```

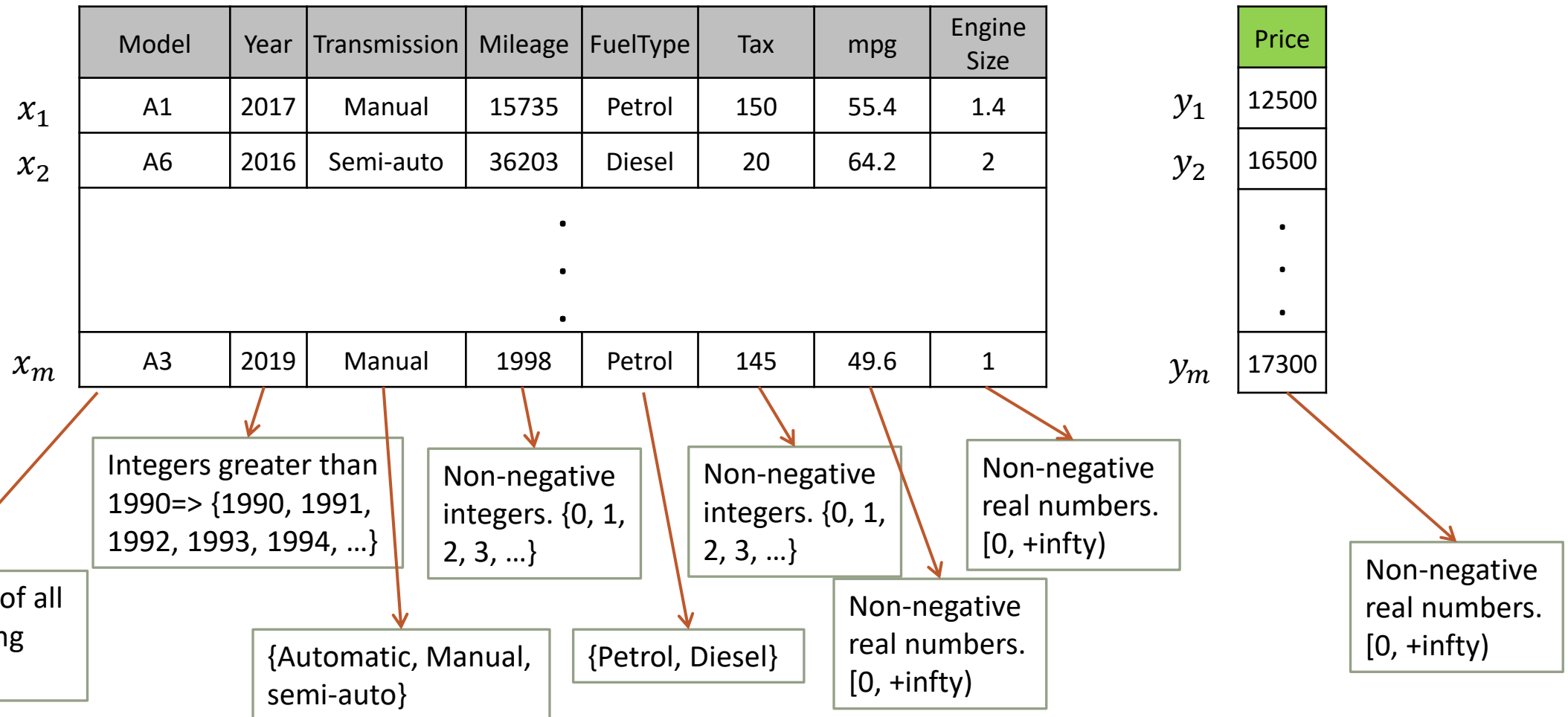
Practical Consideration – Data Preprocessing

Dataset Feature Pre-processing [1]

- Before training an ML algorithm, the labelled dataset is **pre-processed** and prepared for input based on the requirements of the algorithm.
- Some algorithms require that all columns are numbers. So how do we change string columns?
- If the string column only has a finite set of possible values, then you can map each string value to a unique number.

Dataset Feature Pre-processing [2]

Suppose this is $m=1000$ labelled examples. You can see it has $n=8$ feature dimensions. Can you define suitable domains for the feature columns and a range for the output?



Dataset Feature Pre-processing [3]

	Model	Year	Transmission	Mileage	FuelType	Tax	mpg	Engine Size		Price
x_1	A1	2017	Manual	15735	Petrol	150	55.4	1.4	y_1	12500
x_2	A6	2016	Semi-auto	36203	Diesel	20	64.2	2	y_2	16500
				•						•
				•						•
				•						•
x_m	A3	2019	Manual	1998	Petrol	145	49.6	1	y_m	17300

The Model column takes on values from a finite set/list of all car models being sold. We can map each unique model to unique integer:

BMW $\rightarrow$ 1, Accord $\rightarrow$ 2, A1 $\rightarrow$ 3, A2 $\rightarrow$ 4, A3 $\rightarrow$ 5, A4 $\rightarrow$ 6, A5 $\rightarrow$ 7, A6 $\rightarrow$ 8

etc.

Dataset Feature Pre-processing [4]

	Model	Year	Transmission	Mileage	FuelType	Tax	mpg	Engine Size		Price
x_1	3	2017	Manual	15735	Petrol	150	55.4	1.4	y_1	12500
x_2	8	2016	Semi-auto	36203	Diesel	20	64.2	2	y_2	16500
			.	.	.					.
			.	.	.					.
			.	.	.					.
x_m	5	2019	Manual	1998	Petrol	145	49.6	1	y_m	17300

- The Transmission column takes on 3 values {Automatic, Manual, Semi-auto} which we can map to 0, 1, 2 (or any three numbers).
- However, mapping them to numbers implies there is an ordering between Automatic, Manual, and Semi-auto. Even if there is an ordering like in {small, medium, large} how far is the ordering distance?
- A better mapping would be one-hot encoding; each value is mapped to a unique bit-vector.
- Automatic $\rightarrow$ [1, 0, 0], Manual $\rightarrow$ [0, 1, 0], Semi-auto $\rightarrow$ [0, 0, 1] there is no ordering, but rather an on-off bit to indicate the transmission type.

Dataset Feature Pre-processing [5]

	Model	Year	Automatic_Transmission	Manual_Transmission	Semi-auto_transmission	Mileage	FuelType	Tax	mpg	Engine Size	
x_1	3	2017	0	1	0	15735	Petrol	150	55.4	1.4	y_1
x_2	8	2016	0	0	1	36203	Diesel	20	64.2	2	y_2
					.						.
					.						.
					.						.
x_m	5	2019	1	0	0	1998	Petrol	145	49.6	1	y_m



The FuelType column takes on 2 values {Petrol, Diesel} which we can map to 0 and 1 which is equivalent to one-hot encoding [1, 0] and [0,1]. Since both mappings are on-off bits.

Dataset Feature Pre-processing [6]

	Model	Year	Automatic_Transmission	Manual_Transmission	Semi-auto_transmission	Mileage	FuelType	Tax	mpg	Engine Size	
x_1	3	2017	0	1	0	15735	0	150	55.4	1.4	y_1
x_2	8	2016	0	0	1	36203	1	20	64.2	2	y_2
					.						.
					.						.
					.						.
x_m	5	2019	1	0	0	1998	0	145	49.6	1	y_m

Price
12500
16500
.
.
.
17300

Now all inputs (features) and outputs are numbers.

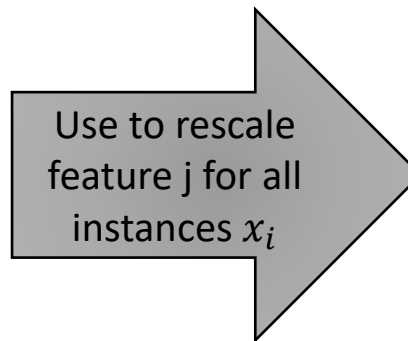
Note pre-processing is an involved topic with many techniques, we will not discuss them further. You can go from here to learn more on your own.

Dataset Feature Pre-processing [6]: Feature rescaling/normalization

- Some algorithms *benefit* from having the **continuous** input features on the same range of values. Like for k -nearest neighbors, linear regression, logistic regression, neural networks.
- Import notes:
 - Any rescaling technique you use should be based on training set T only.
 - You use the training set to derive the *parameters* of the rescaling technique.
 - Those *same* parameters are then used for rescaling training, validation, test, and new instances.
- Min-max Scaler (Normalization)**: rescales the feature j to be in a range $[a, b]$ (typically $[0,1]$).

$$\min_j := \arg \min_{x_i \in T} x_{ij}$$

$$\max_j := \arg \max_{x_i \in T} x_{ij}$$

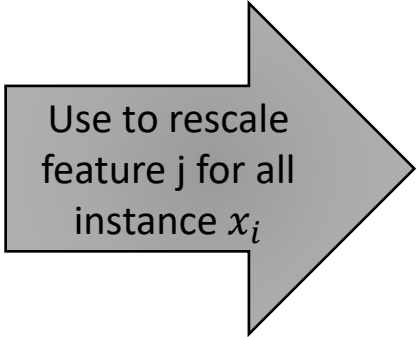


$$x_{ij} := \frac{x_{ij} - \min_j}{\max_j - \min_j} (b - a) + a$$

Note min
and max of
training set.

Dataset Feature Pre-processing [7]: Feature rescaling/normalization

- **Standard Scaler**: rescales the feature j to be centered around 0 and with 1 standard deviation (0 mean and 1 standard deviation).

$$\hat{\mu}_j := \frac{1}{|T|} \sum_{x_i \in T} x_{ij}$$
$$\hat{\sigma}_j := \sqrt{\frac{1}{|T| - 1} \sum_{x_i \in T} (x_{ij} - \hat{\mu}_j)^2}$$


Use to rescale feature j for all instance x_i

$$x_{ij} := \frac{x_{ij} - \hat{\mu}_j}{\hat{\sigma}_j}$$

Note mean and deviation compute from training set

- Primary reason we rescale numeric features is to put them on relatively the same scale so that they can be compared. Example, we have feature 1 with range $[-1000, 1000]$ and feature 2 with range $[0.05, 0.8]$.
- The output label in regression tasks can also be rescaled for easier training of some algorithms. More on this in later lectures.

Dataset Feature Pre-processing [8]

	Model	Year	Automatic_Transmission	Manual_Transmission	Semi-auto_transmission	Mileage	FuelType	Tax	mpg	Engine Size		Price
x_1	3	2017	0	1	0	15735	0	150	55.4	1.4	y_1	12500
x_2	8	2016	0	0	1	36203	1	20	64.2	2	y_2	16500
					.							.
					.							.
					.							.
x_m	5	2019	1	0	0	1998	0	145	49.6	1	y_m	17300

- We commonly apply feature scaling for continuous features. So, for this dataset, we would apply feature scaling to: Year, Mileage, Tax, mpg, and Enginesize.
- The feature Model can also be rescaled, but it would be better to use one-hot encoding to remove ordering.
- Finally, Price can also be rescaled if the values have a massive range that would result in issues when training neural networks [advanced topic]. So, for most problems there is no need to rescale the output.

Min-Max and Standard Scaler Example

$$x_{ij} := \frac{x_{ij} - \min_j}{\max_j - \min_j}(b - a) + a$$

$$x_{ij} := \frac{x_{ij} - \hat{\mu}_j}{\hat{\sigma}_j}$$

Feature
24.32
18.44
-58.51
-33.04
16.96
88.01
-3.3
65.99
6.82
76.64

	Column stats
mean	20.23
stdev	46.75
min	-58.51
max	88.01

Min-Max Scaled Feature
0.57
0.53
0.00
0.17
0.52
1.00
0.38
0.85
0.45
0.92

	Column stats
mean	0.54
stdev	0.32
min	0
max	1

Standard Scaled Feature
0.09
-0.04
-1.68
-1.14
-0.07
1.45
-0.50
0.98
-0.29
1.21

	Column stats
mean	0
stdev	1
min	-1.68
max	1.45

Dataset Feature Pre-processing in python

- Scikit-learn contains great support for preprocessing, evaluation metrics, and general ML utility functions.
- See more info for preprocessing in scikit-learn [here](#).

Here we are using LogisticRegression,
And fitting it on training data.

Here we evaluate performance on
test set

```
import pandas as pd
import numpy as np
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler
from sklearn.metrics import accuracy_score
```

```
# read data from csv
data_df = pd.read_csv('mydata.csv')
X_data = data_df[['feat1', 'feat2', 'feat3', 'feat4', 'feat5']]
y_data = data_df['label']
```

```
# split data into train and test using scikit-learn
# stratify arg used when stratify sampling
X_train, X_test, y_train, y_test = train_test_split(X_data, y_data,
                                                    test_size=0.25, stratify=y_data,
                                                    random_state=42random_state=42)
```

```
# fit scaler on training data
scaler = StandardScaler().fit(X_train)
X_train_scaled = scaler.transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
# fit LR model
lr_model = LogisticRegression()
lr_model.fit(X_train_scaled, y_train)
```

```
# evaluate on test data
y_pred_test = lr_model.predict(X_test_scaled)
test_acc = accuracy_score(y_test, y_pred_test)
print(f'Test accuracy is {test_acc}')
```

ML python libraries

- numpy: linear algebra library for operating on vectors and matrices.
- pandas: library for manipulating datasets in easy to view format.
- scikit-learn: popular machine learning library with lots of ML algorithms.
- tensorflow: popular library for creating computational graphs. Many ML algorithms are computational graphs. Useful for other domains like optimization.
- keras: popular deep learning library. Recently incorporated into tensorflow.
- pytorch: popular deep learning library. Recently more popular than keras.
- matplotlib: popular plotting library for visualizing data or results.
- seaborn: another popular plotting library but with better visuals.
- gym: popular reinforcement learning library with lots of environments for testing your RL algorithms. Most likely not needed for the course.

scikit-learn algorithm cheat-sheet

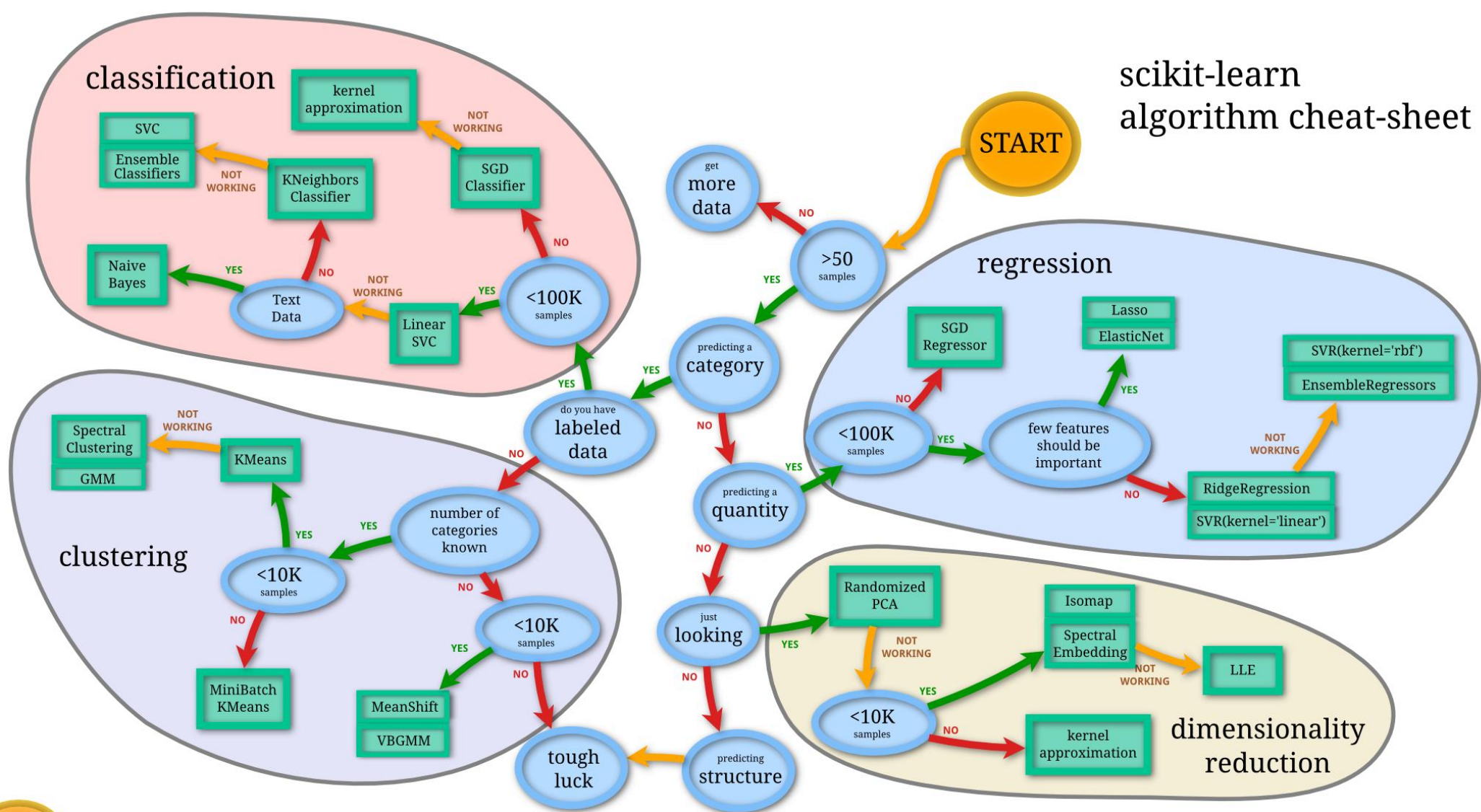


Image from: https://scikit-learn.org/stable/tutorial/machine_learning_map/index.html